

Documentation

None

MNEMOI Lina, GOURDETTE Beevery

None

Table of contents

1. Présentation	3
2. Interface PolybDB	4
2.1 1. Fichiers de sortie	4
2.2 2. Lancement de l'interface	4
2.3 3. Présentation des pages de l'interface	4
2.4 4. Écrire une requête SPARQL	5
3. Architecture	7
3.1 Structure du code	7
3.2 Initialisation RDF	7
3.3 Dictionnaire et valeurs globales	8
3.4 Fonctions annexes	9
3.5 Parseurs	11
3.6 Fonction principale	11
4. Exploitation des données	12
4.1 Rappel	12
4.2 Classes	12
4.3 Propriétés et Relations	13
4.4 Exemple d'instance RDF	14
5. Guide d'implémentation	15
5.1 Base de données	15

1. Présentation

Cette documentation a pour objectif d'offrir tous les outils pour manipuler plus aisément les différentes bases de données contenant les polypes, notamment l'ajout de nouvelles base, une introduction sur le langage de requêtes SPARQL ainsi que les classes et propriétés utilisées durant la création de la base de donnée généralisée.

2. Interface PolybDB

L'interface PolybDB permet de faciliter l'écriture et le lancement de requêtes sur la base de données.

2.1 1. Fichiers de sortie

Une fois la requête exécutée, deux boutons apparaissent dans les résultats : **Export JSON** et **Export CSV**. Ils permettent de récupérer l'ensemble des résultats de la requête dans un fichier exporté.

2.1.1 Exemples de formats :

- **JSON** : exemple pour deux images.
- **CSV** : exemple pour deux images.

2.2 2. Lancement de l'interface

Il faut extraire l'ensemble des fichiers contenus dans `PolypDB.zip` et connaître le chemin vers la base Turtle (`NOM.ttl`). Par défaut, on considère qu'ils sont dans le même dossier.

2.2.1 Contenu de PolypDB.zip :

- Le fichier `index.html`
- Le fichier `main.js`
- Le fichier `preload.js`
- Le dossier `node_modules`

2.2.2 Procédure :

1. Ouvrir l'invite de commande.
2. Se placer dans le bon dossier.
3. Lancer la commande : `python -m http.server 8080`.
4. Ouvrir dans le navigateur : `http://localhost:8080`.

2.3 3. Présentation des pages de l'interface

L'interface est composée de quatre pages et d'un historique de recherche.

2.3.1 3.1 Query Builder

Cette page permet de générer des requêtes. La requête par défaut permet de renvoyer toutes les images avec leur ID, chemin, diagnostic et type d'annotateur.

2.3.2 3.2 Explore

Offre une vue d'ensemble de la base via des statistiques : - Nombre d'images par dataset. - Distribution des diagnostics (et diagnostics CONECT). - Distribution des annotateurs. - Distribution des méthodes de lumière et colorations utilisées. - Distribution des éléments manquants. - Distribution des formats d'images. - Sommaire : nombre total d'images, diagnostics, datasets, classifications et images avec infos patient.

2.3.3 3.2 Free Query

Permet d'écrire sa propre requête manuellement pour les fêrus de SPARQL.

2.3.4 3.3 Documentation

Documentation complète en anglais.

2.3.5 3.4 History

Historique des 60 dernières requêtes exécutées (date, heure, requête, nombre de résultats). Trois boutons : * **Load in Free Query** : Charger la requête dans la page Free Query. * **Exclude these images** : Ajoute une ligne `FILTER` dans le SPARQL pour exclure les identifiants sélectionnés. * **Get more Data** : Permet de parcourir les images de la requête pour sélectionner des informations supplémentaires et enrichir le résultat originel.

2.4 4. Écrire une requête SPARQL

Une requête SPARQL est composée de trois blocs : **SELECT**, **WHERE** et **FILTRES**. Par défaut, chaque objet est contenu dans un IRI. Pour renvoyer l'ID unique, on utilise :

```
BIND(STRAFTER(STR(?img), "#") AS ?img_id)
```

2.4.1 Exemple de requête simple :

```
SELECT ?img_id
WHERE {
  BIND(STRAFTER(STR(?img), "#") AS ?img_id)
}
LIMIT 100
```

Cette requête renvoie les id des 100 premières images de la base de données.

2.4.2 4.1 Le bloc SELECT

Définit les attributs à retourner (ex: `?img_id`).

2.4.3 4.2 Le bloc WHERE

Définit les conditions et les liens entre entités. C'est ici que l'on retrouve l'ensemble des filtres que l'on veut appliquer aux images de la database renvoyées. La page **Query Builder** permet de les écrire automatiquement.

Explication des filtres de base :

Filtrer par database :

```
?img :belongsToDataset db_01 . # avec db_01 l'identifiant de la database souhaitée
```

Filtrer par diagnostic :

```
?img :hasDiagnosis ?diagClass .
?diagClass rdfs:subClassOf* :Polyp . # avec Polyp le diagnostic recherché
```

Filtrer par annotateur :

```
?diagClass :annotatedBy ?ann .
?ann a MedicalExpert .
```

Filtrer par type de lumière :

```
?img :hasLightProcessing ?light_process
```

Filtrer par annotation disponible :

```
?img :hasBoundingBox ?bb_img .  
?img :hasSegmentedImage ?si_img .
```

Filtrer par data manquant (pour savoir dans quelles images on doit chercher les informations) :

Exclure les résultats d'une requête précédente :

2.4.4 4.3 Les filtres de sélection

- **LIMIT N** : Restreint le nombre de résultats pour accélérer le traitement.
- **FILTER** : Permet d'exclure ou de cibler des valeurs spécifiques.

3. Architecture

Afin de mieux comprendre comment la structure du projet, cette section est une vitrine des différentes fonction utilisées pour le parseur ainsi que les noms de classes, sous-classes, propriétés ainsi que les hiérarchies pour pouvoir mieux ajouter de nouvelles bases de données par la suite et pouvoir faire des requêtes fonctionnelles avec les noms adaptés.

3.1 Structure du code

Le code se sépare en trois axes principaux :

- **Initialisation du graphe** : Création du graphe contenant toutes les images.
- **Dictionnaires et valeurs globales** : Afin d'avoir un accès simple aux différentes URI utilisées, réutilisables pour chaque base de données. Contient également les outils d'unifications des images notamment les identifiants de chaque image ou de polypes.
- **Fonctions annexes** : Fonctions utilisées par les parseurs pour automatiser la création des entités (`create_image`, `create_diagnosis`, `create_patient` ...).
- **Parseurs** : Fonction individuelle à chaque base de données pour extraire les informations de chaque image afin de créer les relations liées à celle-ci devant être ajoutées au graphe.
- **Fonction main** : Parcours de chaque base de données et génération finale du fichier Turtle

3.2 Initialisation RDF

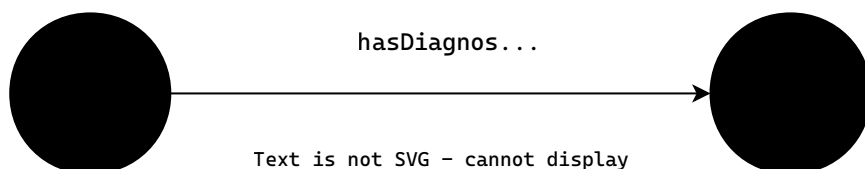
3.2.1 Format

Le graphe des données est stocké dans un fichier RDF sous le format turtle. Les informations sont stockées sous la forme de **triplets** (*sujet, predicats, objet*). La sémantique RDF est détaillée dans le [guide RDF](#). pour plus d'informations. Toutefois, il est important de noter que par la suite, cette sémantique est celle utilisée pour ajouter dans le graphe g :

```
g = Graph()
EX = Namespace("http://example.org/polyps#")
g.bind("ex", EX)
```

3.2.2 Ajout d'une relation

```
g.add((img_uri, EX.hasDiagnosis, diag_uri))
```



3.2.3 Clôture du graphe

Dans le main, le graphe n'est finalisé qu'une fois que l'instruction `g.serialize("EX.ttl", format="turtle")` est exécutée, avec en paramètre le nom du fichier turtle et le format souhaité.

3.3 Dictionnaire et valeurs globales

Deux raisons principales poussent à avoir des dictionnaires ou valeurs globales : la première étant de simplifier la conversion du nom littéral rencontré en URI utilisable par le graphe et les requêtes. D'autre part, nous souhaitons unifier les bases de données en donnant à chaque image un identifiant unique mais en donner également aux patients, polypes, annotateurs et diagnostics afin de lier plus simplement les différents documents entre eux sans rencontrer de conflits notamment avec les noms.

3.3.1 Mapping

Ces dictionnaires permettent de faire une conversion unifiée par tous les parseurs. Nous recommandons d'ajouter si besoin les nouvelles écritures rencontrées dans les dictionnaires suivants afin de mettre à jour automatiquement pour les anciennes bases de données déjà ajoutées mais également pour s'assurer de ne pas faire de doublons d'informations.

Dictionnaire d'annotateurs lorsqu'une base de donnée contient plusieurs annotateurs jugeant la même image. C'est le cas pour la base ``Colonoscopic Dataset``

```
ANNOTATOR_ROLE_MAP = {
  "EXPERT 1": ("MedicalExpert", "Expert 1 COLONOSCOPY"),
  "EXPERT 2": ("MedicalExpert", "Expert 2 COLONOSCOPY"),
  "EXPERT 3": ("MedicalExpert", "Expert 3 COLONOSCOPY"),
  "EXPERT 4": ("MedicalExpert", "Expert 4 COLONOSCOPY"),
  "BEGINNER 1": ("Beginner", "Beginner 1 COLONOSCOPY"),
  "BEGINNER 2": ("Beginner", "Beginner 2 COLONOSCOPY"),
  "BEGINNER 3": ("Beginner", "Beginner 3 COLONOSCOPY"),
}
```

Le dictionnaire `SUBTYPE_MAP` est le dictionnaire clé pour la gestion des diagnostics. Certaines bases de données peuvent avoir des écritures austères comme par exemple `t + v`. Le dictionnaire permet également de prendre en compte les différentes écritures d'un même diagnostic comme par exemple `hyperplasic` et `hyperplastic`. Le dictionnaire permet également de modifier le diagnostic si l'URI ne correspond plus : par exemple, si par la suite il est nécessaire d'avoir une sous-classe dédiée pour tous les `inflammatory` rencontré, modifié le dictionnaire permet de modifier le diagnostic pour toutes les bases de données sans autres vérifications antérieures pour chaque parseur.

```
SUBTYPE_MAP = {
  "adenoma": EX.Adenoma,
  "hyperplastic": EX.Hyperplastic,
  "hyperplasic": EX.Hyperplastic,
  "sessile": EX.SessileSerratedLesion,
  "serrated": EX.SessileSerratedLesion,
  "tubular": EX.TubularAdenoma,
  "villous": EX.TubularVillousAdenoma,
  "adenomatous": EX.Adenoma,
  "normal": EX.Normal,
  "t + v": EX.TubularVillousAdenoma,
  "tubulovillous": EX.TubularVillousAdenoma,
  "traditional serrated adenoma": EX.SessileSerratedLesion,
  "adenocarcinoma": EX.Cancer,
  "cancer" : EX.Cancer,
  "inflammatory": EX.Other,
  "serrated, hyperplastic": [EX.SessileSerratedLesion, EX.Hyperplastic]
}
```

Indique la location de l'entité présente sur la photo, normale ou anormale, dans le corps.

```
LOCATION_MAP = {
  "cecum": EX.Cecum,
  "rectum": EX.Rectum,
  "colon": EX.Colon,
  "gastric": EX.Gastric,
  "stomach": EX.Stomach,
  "pylorus": EX.Pylorus,
  "esophagus": EX.Esophagus,
  "ileum": EX.Ileum,
  "small bowel": EX.SmallBowel,
  "ileum": EX.Ileum,
}
```

Correspondance entre chaque diagnostic de classification comme `JNET` ou `PARIS` et l'URI.


```

PARIS_MAP = {
    "0-Ip": [EX.Paris_0Ip],
    "0-Ips": [EX.Paris_0Ips],
    "0-lps": [EX.Paris_0Ips],
    "0-Is": [EX.Paris_0Is],
    "0-ls": [EX.Paris_0Is],
    "0-IIa": [EX.Paris_0IIa],
    "0-IIb": [EX.Paris_0IIb],
    "0-IIc": [EX.Paris_0IIc],
    "0-IIa + Is": [EX.Paris_0IIa, EX.Paris_0Is],
    "0-IIa + IIc": [EX.Paris_0IIa, EX.Paris_0IIc],
    "0-IIa /c": [EX.Paris_0IIa, EX.Paris_0IIc],
    "0-IIa + Ip": [EX.Paris_0IIa, EX.Paris_0Ip],
    "0-Ip + Is": [EX.Pari_0Ip, EX.Paris_0Is],
}

JNET_MAP = {
    "1": EX.JNET_Type1,
    "2A": EX.JNET_Type2A,
    "2B": EX.JNET_Type2B,
    "3": EX.JNET_Type3,
}

LST_MAP = {
    "LST-G HT": EX.LST_G,
    "LST-G MN": EX.LST_G,
    "LST-NG PD": EX.LST_NG,
    "LST-NG FT": EX.LST_NG,
}

NICE_MAP = {
    "type 1": EX.NICE_Type1,
    "type 2": EX.NICE_Type2,
    "type 3": EX.NICE_Type3,
}

```

3.3.2 Identifiants et liens

Il est parfois demandé pour certaines bases de données de relier une image à une autre ou à un patient. Les dictionnaires de polypes et de patients permettent de stocker les URI pour pouvoir les retrouver lorsqu'une image est liée à celle ci. Cette méthode est utilisé lorsqu'il n'est pas possible de déduire le chemin de images liées.

```

# utilisé pour avoir un identifiant unique pour chaque image, patient, diagnostic, annotateur et polype.
IMAGE_ID = 1
DIAG_ID = 1
ANNOTATOR_ID = 1
POLYP_ID = 1

# stockage des ID et des URI pour pouvoir faire les liens.
POLYP_DIAGNOSES = {}
POLYP_INSTANCES = {}
PATIENT_REGISTRY = {}

```

3.4 Fonctions annexes

Les fonctions annexes permettent de simplifier les parseurs, certaines de ces fonctions sont utilisées quasi systématiquement comme `create_image`, `add_annotator` et d'autres dans les cas de liaisons d'objets comme `get_polype_instance(polype_name)`.

3.4.1 Création d'image

```

def create_image(root, filename, db_id):
    return img_id, img_uri

```

La fonction `create_image` permet de créer les relations de propriétés basiques de l'image:

- Le type (RDF.type)
- La base de données d'origine (EX.belongsToDataset)
- Le chemin d'origine (EX.hasPath)
- Le format de l'image (EX.hasFormat)
- La taille de l'image en octet (EX.fileSize)
- La largeur de l'image (EX.width)
- La longueur de l'image (EX.height)
- La durée de la vidéo en seconde (EX.duration)

Toutes les propriétés sont détaillées dans le [tableau des propriété](#).

La fonction prends en paramètre le chemin vers le fichier, le nom du fichier et l'ID de son dataset d'origine. La fonction retourne l'ID de l'image créée ainsi que son URI.

3.4.2 Création de diagnostics

```
def create_diagnosis(img_uri, diag_type_uri, annotator_uri):
    return diag_uri
```

Crée les propriétés de base liées à un diagnostic d'une image : Prends en paramètre l'URI de l'image, l'URI du type de diagnostic comme par exemple EX:Polyp et l'URI de l'annotateur de l'image. Retourne l'URI du diagnostic.

- l'annotateur (EX.annotatedBy)
- le diagnostic (EX.hasDiagnosis)

3.4.3 Ajout d'annotateur

```
def add_annotator(annotator_type, label=None, verified=False):
```

Par défaut, toutes les images ont des annotateurs, qu'on estime être des experts médicaux (EX.MedicalExpert). Un annotateur peut être un expert médical, un laboratoire ou un débutant. Le label, None par défaut, est le nom de l'annotateur. Nous estimons possible que dans le futur, le nom du médecin soit disponible, d'où le label. Il est toutefois déjà utile pour préciser quel est le type d'expert médical. La fonction retourne l'URI de l'annotateur.

- la véracité de l'information (EX.isVerified)
- le type d'annotateur

3.4.4 Chargement des bases de données

Afin de simplifier la prise en main, les métadonnées liées aux bases de données sont contenues dans un fichier json dans lesquelles il faut simplement entrer les informations liées à la base

```
def load_databases(json_path):
```

Prends en paramètre le chemin du fichier contenant les informations des bases de données et crée

- le pays de la base (EX:country)
- le nom de la base (EX:name)
- le type d'accès (privé ou public)
- l'ID de la base de données

3.4.5 Gestion des polypes partagés

Dans notre architecture, nous partons du principe **qu'une image correspond à un polype** (ou autre diagnostic). Toutefois, il arrive qu'un même polype soit représenté par plusieurs images. Pour éviter les erreurs lors des test, nous utilisons la fonction `get_polype_instance` avec le nom du polype afin de créer ou récupérer l'URI du polype à ajouter à une nouvelle image.

- Le type (EX.PolypInstance)

3.4.6 Gestion des patients

De même pour les polypes, certaines images sont liées à un unique patient. Il peut être intéressant de stocker l'âge ou le sexe du patient, ou bien de voir d'autre polypes du patient (ou bien le même polype dans un angle différent).

```
def create_patient(patient_code, sex=None, age=None, is_child=False):
```

La fonction ajoute, lorsque possible, les informations passées en paramètre puis retourne l'URI du patient. Nous conseillons par la suite d'ajouter des paramètres supplémentaires facultatifs (comme par exemple, si dans le futur une base de données contient entièrement un registre de patient en rémission).

3.5 Parseurs

Tous les parseurs ont comme paramètres obligatoires

```
def parse_BDD(root, filename, ann_uri):
```

L'objectif est de stocker toutes les informations relatives à l'image : Bien que certaines propriétés sont traçables simplement (taille en octet, durée d'une vidéo...), la plupart des base de données nécessitent d'étudier **individuellement la structure**. Des conseils pour savoir quelles questions se poser et la méthodologie à avoir sont mieux expliquées dans le [guide des bases de données](#). En résumé, la partie parseurs permet pour chaque base de données d'ajouter au graphe g les informations accessibles pour une image. Il est aussi important de noter que certaines base de données nécessitent des fonctions annexes pour la lecture de fichier CSV ou XML par exemple.

3.6 Fonction principale

La fonction `main` est le lieu où l'opération de construction de graphe se produit :

```
ann_kumc = add_annotator("MedicalExpert", "MedicalExpert KUMC")

kumc_root = os.path.abspath(os.path.join(current_dir, "..", "data", "KUMC Dataset", "KUMC Dataset", "Dataset"))
if os.path.exists(kumc_root):
    for root, _, files in os.walk(kumc_root):
        for file in files:
            parse_KUMC(root, file, ann_kumc)
```

En prenant exemple sur l'application d'un parseur sur le dataset KUMC, on voit qu'il est nécessaire de parcourir chaque image. Il faut donc avoir le chemin d'accès à chaque dossier et parcourir en utilisant le parseur sur chaque image. Également, on rappelle que les annotateurs sont par défaut non vérifiés, et qu'il faut les gérer dans cette partie du code principal.

La fonction principale doit se terminer par la [clôture du graphe](#)

4. Exploitation des données

Cette section a pour but de simplifier l'écriture de requêtes en énumérant les différentes classes, propriétés ou objets existants de façon la plus exhaustive possible. Nous invitons à mettre à jour fréquemment lors des changements ou des ajouts de relations.

4.1 Rappel

Les relations RDF se présentent sous la forme (sujet, prédicat, objet). Un sujet doit être une URI et un objet peut être une URI, un littéral, booléen ou même un entier. Les tableaux suivants permettent de reconnaître quelles sont les données à entrer lors de la requête ou de l'ajout d'une base de donnée, afin d'avoir de retrouver correctement les bonnes informations.

4.2 Classes

Classe	Description
<code>ex:Annotator</code>	Entité humaine ou institutionnelle qui annote les images médicales.
<code>ex:MedicalExpert</code>	Sous-classe d'Annotator. Annotateur ayant une expertise médicale reconnue (gastro entérologue, physiologiste).
<code>ex:Beginner</code>	Sous-classe d'Annotator. Annotateur débutant.
<code>ex:Laboratory</code>	Sous-classe d'Annotator. Les annotations d'un laboratoires sont considérées comme ground truth.
<code>ex:Image</code>	Image ou vidéo annotée.
<code>ex:Polyp</code>	Type de diagnostic: l'image ou la vidéo considérée est annotée comme un polype.
<code>ex:Normal</code>	Type de diagnostic: l'image ou la vidéo considérée est annotée comme une image sans maladie.
<code>ex:Database</code>	Base de données source des images.
<code>ex:ClassificationSystem</code>	Type de classification utilisée pour le diagnostic d'une image ou d'une vidéo.
<code>ex:PolypInstance</code>	Polype existant lié à plusieurs images ou vidéos.
<code>ex:LightProcessing</code>	Technique d'imagerie lumineuse utilisée lors de la capture (WLI, NBI).
<code>ex:Patient</code>	Patient lié a une ou plusieurs image, sur lequel on possède des informations (âge, sexe, adulte ou enfant).
<code>exDiagnostic:</code>	Décision d'un annotateur sur une image ou vidéo donnée.
<code>ex:Bounding Box</code>	Contient le contour de la boundingbox lorsque l'image en contient une.

4.2.1 Hiérarchie des Classes

Annotateurs

- `ex:Annotator`
- `ex:MedicalExpert`
- `ex:Beginner`
- `ex:Laboratory`

Diagnostics

- `ex:Diagnostic`
- **Type principaux**
- `ex:Normal`
- `ex:Polyp`
- `ex:Adenoma`
- `ex:TubularAdenoma`
- `ex:TubularVillousAdenoma`
- `ex:Hyperplastic`
- `ex:SessileSerratedLesion`
- `ex:Cancer`
- `ex:Other`
- **Classification de Paris**
- `ex:Paris_0Ip`
- `ex:Paris_0Ips`
- `ex:Paris_0Is`
- `ex:Paris_0IIa`
- `ex:Paris_0IIb`
- `ex:Paris_0IIc`
- **Classification JNET**
- `ex:JNET_Type1`
- `ex:JNET_Type2A`
- `ex:JNET_Type2B`
- `ex:JNET_Type3`
- **Classification LST**
- `ex:LST_G`
- `ex:LST_NG`
- **Classification NICE**
- `ex:NICE_Type1`
- `ex:NICE_Type2`
- `ex:NICE_Type3`

4.3 Propriétés et Relations

Les propriétés définissent les attributs des classes (Propriétés) et les liens entre elles (Relations).

4.3.1 Attributs principaux

Propriété	Domaine	Range	Description
ex:age	ex:Patient	xsd:integer	Âge du patient.
ex:sex	ex:Patient	xsd:string	Sexe du patient ("M" ou "F").
ex:isAdult	ex:Patient	xsd:boolean	indique si le patient est majeur.
ex:expertise	ex:Annotator	xsd:string	Niveau ou domaine d'expertise de l'annotateur.
ex:isVerified	ex:Annotator	xsd:boolean	Indique si le statut de l'annotateur est sûr ou non (par défaut MedicalExpert).
ex:filePath	ex:Image	xsd:string	Chemin d'accès au fichier.
ex:fileSize	ex:Image	xsd:integer	Taille du fichier image en octets.
ex:height	ex:Image	xsd:integer	Hauteur de l'image en pixels.
ex:width	ex:Image	xsd:integer	Largeur de l'image en pixels.
ex:hasFormat	ex:Image	xsd:string	Format du fichier image (jpg, tif, png ...).
ex:duration	ex:Image	xsd:decimal	en secondes pour les vidéos.
ex:name	ex:Database	xsd:string	Nom officiel de la base de données.
ex:link	ex:Database	xsd:string	URL d'accès vers la source de la base de donnée.
ex:access	ex:Database	xsd:string	Indique comment le type d'accès à une base de donnée ("public" ou "private").

4.3.2 Relations entre entités

Propriété	Domaine	Range	Description
ex:belongsToDataset	ex:Image	ex:Dataset	Relie une image et son dataset originel.
ex:hasDiagnosis	ex:Image	ex:Diagnostic	Relie une image à son diagnostic lié. Une image peut avoir plusieurs diagnostics selon le type de classification ou si plusieurs annotateurs.
ex:hasPolypInstance	ex:Image	ex:PolypInstance	Relie une instance de polype à plusieurs images qui le représente.
ex:annotatedBy	ex:Diagnostic	ex:Image	Indique de quel annotateur provient la décision.

4.4 Exemple d'instance RDF

Voici comment une instance est représentée selon la documentation :

```
ex:img_0000037954 a ex:Image;
  ex:belongsToDataset ex:db_11;
  ex:fileSize 96781;
  ex:hasBoundingBox ex:bbox_img_0000037954;
  ex:hasDiagnosis ex:diag_0000035506;
  ex:hasFormat "jpg";
  ex:hasPath "data/PolypsSet/Polyps5et/train2019/Image/386.jpg";
  ex:hasPolypInstance ex:polyp_023460;
  ex:height 464;
  ex:width 592.
```

5. Guide d'implémentation

Cette section a pour objectif d'assurer une extension simplifiée du dataset dans le future, ainsi que des explications pour une utilisation actuelle du langage de requête SPARQL.

5.1 Base de données

5.1.1 Premiers pas

Commencer par modifier le fichier `databases.json` pour ajouter la nouvelle base de donnée en veillant bien à ne pas créer de clé double. Nous conseillons de garder les identifiants de base de données dans un ordre croissant.

Exemple avec le dataset Endoscopy

```
"db_03": {
  "name": "Endoscopy Campus Dataset",
  "country": "Germany",
  "hospital": null,
  "link": "https://www.endoscopy-campus.com/en/classifications/polyp-classification-nice/",
  "access": "public",
  "require_request": false
},
```

Pour la suite, certaines bases de données offrent les informations sur les images dans des fichiers annexes (XML, EXCEL, CSV...). Nous conseillons de débiter par détecter ces fichiers avant de débiter et de créer les fichiers annexes de lecture associées.

5.1.2 Parcours des données

Dans la fonction principale, créer l'annotateur ou les annotateurs associés ainsi qu'une variable de chemin vers les dossiers contenant les images.

```
ann_cp = add_annotator("MedicalExpert", "MedicalExpert CP-CHILD")

cp_root = os.path.abspath(os.path.join(current_dir, "..", "data", "CP_CHILD_DATASET", "CP_CHILD_DATASET", "CP-CHILD-A"))
if os.path.exists(cp_root):
    for root, _, files in os.walk(cp_root):
        for file in files:
            parse_CP_Child(root, file, ann_cp)
```

Nous rappelons que les annotateurs sont par défaut considérés comme **non vérifié**. Il est donc nécessaire à cette étape de vérifier les sources de la base de données comme par exemple sur le site ou sur un fichier annexe pour savoir. Sinon, par défaut se créer un annotateur `MedicalExpert` avec comme label `MedicalExpert + NOM DATASET`. Dans le futur, si l'information de l'annotateur est vérifié, il suffit d'ajouter lors de l'appel de la fonction `add_annotator` le paramètre `is_verified=True`.

5.1.3 Traitement des images

Chaque image est pour la plupart du temps, traitée individuellement lors d'un parcours d'image dans chaque dossier. Chaque base de données pouvant avoir des structures très différentes, chaque parseur sera unique. Toutefois, nous souhaitons simplifier les futures implémentations en guidant la prise en main des parseurs existants.

Implémentation récurrente

Nous allons principalement utiliser les **fonctions annexes** de notre architecture qui permettent d'établir un squelette de parseur.

Au cours du parseur, une des étapes essentielle est de **se créer l'URI d'une image** à l'aide la fonction

```
imd_id, img_uri = create_image(root, filename, "db_00")
```

La variable `img_uri` permet donc d'ajouter dans la base de données, les propriétés ou les relations de l'image inaccessibles avec `create_image`.

Exemple

```
g.add((img_uri, RDF.type, EX.OriginalImage))
g.add((img_uri, EX.hasSegmentedImage, mask_uri))
g.add((img_uri, EX.hasPolypInstance, polyp_uri))
```

Nous invitons à revoir le [tableau des relations et des propriétés](#) ainsi que la [signature de la fonction `create\_image`](#) pour voir quelles sont les propriétés et relations possibles à ajouter en dehors de la fonction `create_image`. Cette variable doit être nécessairement créer avant les URI de `EX:PolypInstance` et `EX:Diagnosis` car celles-ci sont dépendantes de l'image.

Par la suite, il est important de déterminer où trouver le diagnostic d'une image (fichier CSV, nom du dossier, nom du fichier, inconnu...). Une image peut avoir plusieurs diagnostics. En effet plusieurs annotateurs peuvent juger une image mais il est également possible qu'une image subisse un diagnostic avec des **classifications différentes**. Il est donc important de vérifier durant l'étude de la base de données si l'un des deux cas, ou si les deux cas, mentionnés sont présent. Les diagnostics sont répétés pour toutes les images d'un même polype.

```
create_diagnosis(img_uri, diag_type, ann_uri)
```

Si la situation de plusieurs polypes se présente, il faut généralement stocker un nom de polype permettant de relier plusieurs images entre elles (à l'aide d'un fichier CSV, dans le nom de l'image...)

```
polyp_uri = get_polyp_instance(base_name)
g.add((img_uri, EX.hasPolypInstance, polyp_uri))
```

Enfin, si les patients existent, les ajouter en utilisant la fonction `create_patient`

```
patient_uri = create_patient(patient_code, row.get("Sex"), row.get("Age"))
```

Enjeux principaux

Pour chaque base de données, nous recommandons de s'assurer que le parseur implémenté répondent bien aux problématiques suivantes:

- L'image a-t-elle été bien implémentée? Si oui, quelles sont les relations et propriété à disposition ? (Diagnostic, BoundingBox, d'autres images présentant le même polype, patient, localisation du polype...)
- Ou se trouve le diagnostic ? Il y a-t-il plusieurs annotateurs? Plusieurs classifications utilisées?
- De quelle manière une image est reliée à ses images ségmentées si existantes? L'instance de polype a-elle bien été créée?
- Quelles informations annexes sont disponibles dans la base de données?

Tableau récapitulatif des bases de données

Dans le futur, si une nouvelle base de donnée doit être ajoutée, nous recommandons de s'inspirer des bases de données déjà implémentées pour simplifier les opérations et en modifiant les parties essentielles

Nom du Dataset	Structure
AIDA-E	Sous-dossiers contenant les images, images originales et segmentées dans le même dossier avec les mêmes noms
BKAI-IGH, Piccolo	Sous-dossiers contenant les images, images originales et segmentées de même nom dans des dossiers différents
Colonoscopic Dataset	Images contenues dans des dossiers, dont les noms correspondent au diagnostics. Diagnostics dans fichier CSV par plusieurs annotateurs vérifiés
CP_Child, Gastrovision, KID, KUMC, Mixed Dataset	Images contenues dans des dossiers, dont les noms correspondent au diagnostics.
Endoscopy Campus Dataset	Images contenues dans le dossier principal. Diagnostic dans le nom du fichier
ERCPMP, Polar cropped	Diagnostic et patients contenus dans un fichier CSV. Plusieurs images et/ou vidéos désignant un même polype. Toutes les images dans un même dossier.
Kvasir Capsule, PolypGen	Images dans un dossier et CSV contenant les coordonnées de la bounding box avec le même nom